

ETAPA 3 - INTEGRADOR

Modelo de datos y persistencia

Institución: Asociación "Todos al Agua" (Necochea)

Integrantes

Acuña Marcos

Fort Lisandro

Luna Franco

Documento técnico - Etapa 3

3.1 Entidades del sistema, con sus atributos y relaciones

A partir del análisis de los wireframes y del menú provisto, se identifican las siguientes entidades, sus atributos principales y las relaciones que permiten estructurar el sistema.

1. Usuario: **id_usuario**, **id_rol**, **nombre_usuario**, **contraseña_hash**, **email**, **foto_perfil_url**, **fecha_creacion**.
2. Rol: **id_rol**, **nombre_rol** (ej. Admin, Encargado_Cantina), **descripcion**.
3. UnidadProductiva: **id_unidad**, **nombre**, **descripcion**, **ubicacion**.
4. Categoría: **id_categoria**, **id_unidad**, **nombre_categoria**, **descripcion**, **nota_adicional** (ej. "Todo con Fritas, Ensalada o Puré").
5. Producto: **id_producto**, **id_categoria**, **nombre**, **descripcion**, **precio** (DECIMAL NULL), **promocion**, **es_sin_tacc**, **disponible**.
6. VarianteProducto: **id_variante**, **id_producto**, **nombre_variante** (ej. "Chica", "Grande"), **precio**.
7. Pedido: **id_pedido**, **id_usuario** (NULL), **nombre_cliente**, **telefono_whatsapp**, **fecha_pedido**, **estado**, **tipo**.
8. DetallePedido: **id_detalle**, **id_pedido**, **id_producto**, **cantidad**, **precio_unitario**.
9. PuntoVenta: **id_punto**, **id_unidad**, **lugar**, **dias_horarios**.
10. MensajeContacto: **id_mensaje**, **nombre_remitente**, **email_remitente**, **area_destino**, **mensaje**, **fecha_envio**.

Relaciones principales

- Un Rol puede estar asociado a muchos Usuarios; cada Usuario posee un único rol.
- Una UnidadProductiva puede contener múltiples Categorías y Puntos de Venta.
- Una Categoría agrupa múltiples Productos.
- Un Producto puede tener múltiples VariantesProducto.
- Un Pedido puede tener múltiples DetallePedido; cada detalle referencia un Producto.
- Un Usuario puede estar asociado a múltiples Pedidos, aunque el usuario de un Pedido puede ser NULL para permitir pedidos de clientes no autenticados.

3.2 Modelo de datos (DDL SQL)

El siguiente DDL define las tablas principales, claves primarias, claves foráneas, restricciones y valores por defecto del modelo relacional.

```
CREATE TABLE Rol (
    id_rol SERIAL PRIMARY KEY,
    nombre_rol VARCHAR(50) NOT NULL UNIQUE,
    descripcion VARCHAR(255)
);

CREATE TABLE Usuario (
    id_usuario SERIAL PRIMARY KEY,
    id_rol INT NOT NULL REFERENCES Rol(id_rol),
    nombre_usuario VARCHAR(50) NOT NULL UNIQUE,
    contraseña_hash VARCHAR(255) NOT NULL,
    email VARCHAR(100) UNIQUE,
    foto_perfil_url VARCHAR(255),
    fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE UnidadProductiva (
    id_unidad SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    ubicacion VARCHAR(150)
);

CREATE TABLE Categoria (
    id_categoria SERIAL PRIMARY KEY,
    id_unidad INT NOT NULL REFERENCES UnidadProductiva(id_unidad) ON DELETE CASCADE,
    nombre_categoria VARCHAR(100) NOT NULL,
    descripcion TEXT,
    nota_adicional VARCHAR(255)
);

CREATE TABLE Producto (
    id_producto SERIAL PRIMARY KEY,
    id_categoria INT NOT NULL REFERENCES Categoria(id_categoria) ON DELETE CASCADE,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT,
    precio DECIMAL(10, 2) NULL,
    promocion VARCHAR(150),
    es_sin_tacc BOOLEAN DEFAULT FALSE,
    disponible BOOLEAN DEFAULT TRUE
);

CREATE TABLE VarianteProducto (
    id_variante SERIAL PRIMARY KEY,
    id_producto INT NOT NULL REFERENCES Producto(id_producto) ON DELETE CASCADE,
    nombre_variante VARCHAR(50) NOT NULL,
    precio DECIMAL(10, 2) NOT NULL
);

CREATE TABLE Pedido (
    id_pedido SERIAL PRIMARY KEY,
    id_usuario INT REFERENCES Usuario(id_usuario) ON DELETE SET NULL,
    nombre_cliente VARCHAR(100) NOT NULL,
    telefono_whatsapp VARCHAR(30),
    fecha_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(30) DEFAULT 'Pendiente',
    tipo VARCHAR(50) NOT NULL
);

CREATE TABLE DetallePedido (
    id_detalle SERIAL PRIMARY KEY,
    id_pedido INT NOT NULL REFERENCES Pedido(id_pedido) ON DELETE CASCADE,
    id_producto INT NOT NULL REFERENCES Producto(id_producto),
    cantidad INT NOT NULL CHECK (cantidad > 0),
    precio_unitario DECIMAL(10, 2) NOT NULL
);
```

3.3 Análisis comparativo de alternativas de persistencia

Alternativa	Ventajas / características	Desventajas / consideraciones
Base de Datos Relacional (PostgreSQL)	Excelente integridad ACID y consistencia estricta. Soporta búsquedas de texto plano para el buscador R4 y relaciones estructuradas.	Requiere un esquema definido y mayor planificación inicial.
Base de Datos Documental (MongoDB)	Flexible ante esquemas variables y documentos con estructuras diferentes.	Requiere validaciones por código para mantener la integridad de pedidos y roles.
Sistema de archivos (JSON Flat-Files / SQLite)	Adecuado para prototipos simples y proyectos de baja complejidad.	No escala bien ante múltiples solicitudes concurrentes o transacciones simultáneas.

3.4 Decisión y fundamentación

Se elige PostgreSQL como sistema de persistencia. Esta elección garantiza la integridad referencial entre usuarios, roles y productos, permitiendo mantener una estructura de datos consistente y claramente definida.

La elección de PostgreSQL se considera temporal. En esta etapa resulta conveniente trabajar con un modelo relacional debido a que los datos presentan una estructura definida. Sin embargo, si en una etapa posterior los datos dejan de mantener una estructura rígida y comienzan a presentar características propias de un modelo NoSQL, se podrá evaluar la migración o incorporación de una alternativa de persistencia documental.

A nivel de la API, se respetará la persistencia y el modelo de datos creado previamente en PostgreSQL. La lógica de la aplicación se construirá sobre la estructura definida, evitando acoplar directamente las vistas a la base de datos.

Para las views se propone utilizar el patrón de inyección de dependencias. De esta manera, las vistas no dependerán directamente de PostgreSQL ni de una implementación concreta de la base de datos, sino de la estructura y de las abstracciones definidas previamente. Esto permite mantener una arquitectura más desacoplada y facilita una futura modificación del mecanismo de persistencia sin tener que modificar la lógica de presentación.

3.5 Juego de datos de prueba

Los siguientes datos de prueba están basados en el JSON provisto y permiten verificar la carga inicial de roles, usuarios, unidad productiva, categorías, productos, variantes y pedidos.

```
-- Roles y Usuarios
INSERT INTO Rol (id_rol, nombre_rol, descripcion) VALUES
(1, 'Administrador', 'Acceso total al sistema'),
(2, 'Encargado_Cantina', 'Gestion de El Muelle Cantina');

INSERT INTO Usuario (id_usuario, id_rol, nombre_usuario, contraseña_hash, email) VALUES
(1, 1, 'admin_mando', '$2b$10$e83921...', 'admin@todosalagua.org'),
(2, 2, 'cantina_admin', '$2b$10$k32029...', 'cantina@todosalagua.org');

-- Unidad Productiva y Categorías
INSERT INTO UnidadProductiva (id_unidad, nombre, ubicacion) VALUES
(1, 'El Muelle - Cantina Inclusiva', 'Club Náutico Necochea');

INSERT INTO Categoria (id_categoria, id_unidad, nombre_categoria, nota_adicional) VALUES
(1, 1, 'Pastas', NULL),
(2, 1, 'Celiacos', NULL),
(3, 1, 'Pizzas', NULL),
(4, 1, 'Nuestros Platos', 'Todo con Fritas, Ensalada o Puré'),
(5, 1, 'Postres', NULL);

-- Productos del JSON
INSERT INTO Producto (id_producto, id_categoria, nombre, descripcion, precio, promocion, es_sin_tacc) VALUES
(1, 1, 'Sorrentinos', 'Jamón y Queso - Verdura - Calabaza', 8500.00, NULL, FALSE),
(2, 2, 'Pizzeta con muzza', NULL, 13500.00, NULL, TRUE),
(3, 2, 'Sandwiches', NULL, NULL, NULL, TRUE), -- Precio NULL a consultar
(4, 3, 'Muzzarella', NULL, NULL, NULL, FALSE),
(5, 4, 'Pollo a la Pastora', 'Pechuga Rebozada c/ verduras - Jamón y queso - Salsa y Papas', 16500.00, NULL, FALSE),
(6, 5, 'Flan', NULL, 3500.00, NULL, FALSE),
(7, 5, 'Adicional Dulce', NULL, 1100.00, 'Adicional 2 x $1500', FALSE);

-- Variantes para Pizzas
INSERT INTO VarianteProducto (id_variante, id_producto, nombre_variante, precio) VALUES
(1, 4, 'Chica', 8000.00),
(2, 4, 'Grande', 12100.00);

-- Pedido de prueba
INSERT INTO Pedido (id_pedido, id_usuario, nombre_cliente, telefono_whatsapp, estado, tipo) VALUES
(1, NULL, 'Carlos Gómez', '2262554433', 'Pendiente', 'Pedido');

INSERT INTO DetallePedido (id_detalle, id_pedido, id_producto, cantidad, precio_unitario) VALUES
(1, 1, 1, 2, 8500.00),
(2, 1, 5, 1, 16500.00);
```

Fin del documento

Etapa 3 - Integrador | Asociación "Todos al Agua" (Necochea)